Getting Started: Two-Sample Tests for Distribution Shifts

The problem

A model trained on some data distribution eventually sees production data that has **drifted** away from it. Detecting this is a **two-sample hypothesis test**: given a reference sample and a new sample, decide whether they come from the same distribution.

Standard practice reaches for **nonparametric** tests (MMD, classifier-based two-sample tests) because you don't know the true distribution. But that safety costs statistical power and scales badly with dimension and sample size. Often, real data does approximately follow some known structure — and a **classical parametric test** built on that assumption (even if it's only approximately true) can detect shifts with far fewer samples and lower latency than a generic nonparametric catch-all.

Goal of this project: quantify when and how much classical methods can beat nonparametric ones at this task, and push toward a theoretical (not just empirical) result. MMD / C2ST-style methods are baselines to compare against, not the main focus.

Start here

Read: Rabanser, Günnemann & Lipton, "Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift" (NeurIPS 2019). It frames shift detection as: reduce dimensionality, then run a two-sample test. Its key surprise — a crude combination of simple per-dimension tests beat a fancy multivariate kernel test — is direct evidence for this project's thesis.

Learning path (no prior hypothesis-testing background needed)

Build the concepts by implementing them, in this order:

1. **Two-sample z/t-test** for a mean shift — learn null/alternative hypotheses, p-values, significance level, Type-I error, power.
2. **Two-sample KS test** (nonparametric) + a **permutation test** framework (general-purpose machine you'll reuse throughout).
3. **Bonferroni correction** for combining many per-dimension tests — reproduce a small slice of the reference paper's pipeline on MNIST or synthetic data and confirm its result yourself.

4. **Generalized Likelihood Ratio Test (GLRT)** as the general recipe for parametric testing — verify its theoretical (chi-squared) behavior empirically before trusting it on real problems.

Workflow

1. Finish the learning path above through hands-on reproduction.
2. Run a broad empirical sweep: inject known, controlled shifts into synthetic data and compare classical parametric tests against nonparametric baselines across sample size and shift magnitude.
3. Narrow to 1–2 promising directions (see below) and push one toward an actual theoretical result, not just a benchmark number.

Directions to explore

Pick 1–2 to go deep on; treat the rest as backup:

- **Efficiency comparison** — how much fewer samples does a classical test need to match a nonparametric test's power, when its assumption roughly holds?
- **Misspecification envelope** — how wrong can the parametric assumption be before the classical test's advantage disappears or breaks down?
- **Detection latency** — does a sequential/online parametric test detect shift faster than repeatedly re-running a nonparametric batch test?
- **Smarter combination rules** — can you beat plain Bonferroni combination of per-dimension tests by exploiting assumed/estimated structure across dimensions?
- **Better compression** — can you design a dimensionality-reduction step that beats PCA / deep-learning-based reductions for shift detection?
- **Random-projection KS** — instead of running KS per raw dimension, run it over random linear combinations of dimensions, combined via Bonferroni. Does that detect more shifts?

Ground rules

- Start empirical, end theoretical — every direction should end with a provable (or at least derivable) statement, not just a plot.
- Avoid reaching for MMD/C2ST or other heavy nonparametric machinery as a first move — that's the "lazy" default this project is trying to beat.
  Use them later only as baselines.